Project Structure Blueprint

Christmas Tracker iOS Application

Version: 1.0
Date: February 3, 2026
Status: Approved

Overview

This document defines the complete project structure for the Christmas Tracker iOS application, including folder hierarchy, file naming conventions, module organization, and best practices for maintaining a clean, scalable codebase.

---

1. Root Project Structure

```
ChristmasTracker/
├── ChristmasTracker/              # Main app target
├── ChristmasTrackerTests/         # Unit tests
├── ChristmasTrackerUITests/       # UI tests
├── ShareExtension/                # Share extension target
├── Config/                        # xcconfig files
├── Scripts/                       # Build scripts, code generation
├── .github/                       # GitHub Actions workflows
├── ChristmasTracker.xcodeproj     # Xcode project
├── .gitignore
├── README.md
└── LICENSE
```

---

2. Main App Target Structure

```
ChristmasTracker/
├── App/
│   ├── ChristmasTrackerApp.swift          # App entry point
│   ├── AppDelegate.swift                  # App lifecycle
│   └── SceneDelegate.swift                # Scene management
│
├── Core/
│   ├── Configuration/
│   │   ├── AppConfiguration.swift         # xcconfig reader
│   │   └── Environment.swift              # Environment detection
│   │
│   ├── Networking/
│   │   ├── APIClient.swift                # Main API client actor
│   │   ├── NetworkCache.swift             # Session-scoped cache
│   │   ├── ErrorInterceptor.swift         # Global error handling
│   │   └── HTTPMethod.swift               # HTTP method enum
│   │
│   ├── Security/
│   │   ├── KeychainService.swift          # Keychain operations
│   │   ├── DeviceFingerprintService.swift  # Device identification
│   │   └── BiometricService.swift         # Face ID/Touch ID
│   │
│   └── Logging/
```

```
│       ├── Logger.swift                    # Main logger actor
│       ├── LogEvent.swift                  # Log event model
│       ├── LogDestination.swift            # Protocol
│       ├── OSLogDestination.swift          # iOS native logging
│       ├── ConsoleDestination.swift        # Debug console
│       ├── NewRelicDestination.swift       # New Relic integration
│       ├── BackendDestination.swift        # Fallback backend
│       └── PIIFilter.swift                 # PII redaction
│
├── Features/
│   ├── Authentication/
│   │   ├── Views/
│   │   │   ├── LoginView.swift
│   │   │   ├── RegisterView.swift
│   │   │   └── BiometricSetupView.swift
│   │   ├── Services/
│   │   │   └── AuthenticationService.swift
│   │   ├── Repositories/
│   │   │   ├── AuthRepository.swift        # Protocol
│   │   │   ├── NetworkAuthRepository.swift
│   │   │   └── MockAuthRepository.swift
│   │   └── Models/
│   │       ├── LoginRequest.swift
│   │       ├── LoginResponse.swift
│   │       └── AuthError.swift
│   │
│   ├── Dashboard/
│   │   ├── Views/
│   │   │   ├── DashboardView.swift
│   │   │   ├── ListCard.swift
│   │   │   └── StatsCard.swift
│   │   ├── Services/
│   │   │   └── DashboardService.swift
│   │   └── Models/
│   │       └── DashboardData.swift
│   │
│   ├── Lists/
│   │   ├── Views/
│   │   │   ├── ListDetailView.swift
│   │   │   ├── CreateListView.swift
│   │   │   ├── EditListView.swift
│   │   │   ├── ListPermissionsView.swift
│   │   │   └── InvitationView.swift
│   │   ├── Services/
│   │   │   └── ListService.swift
│   │   ├── Repositories/
│   │   │   ├── ListRepository.swift
│   │   │   ├── NetworkListRepository.swift
│   │   │   └── MockListRepository.swift
│   │   ├── DataStores/
│   │   │   └── ListDataStore.swift
│   │   └── Models/
│   │       ├── GiftList.swift
│   │       ├── ListMember.swift
│   │       ├── MemberRole.swift
│   │       └── InvitationStatus.swift
│   │
│   ├── Items/
│   │   ├── Views/
│   │   │   ├── ItemDetailView.swift
│   │   │   ├── AddItemView.swift
│   │   │   ├── EditItemView.swift
│   │   │   ├── PurchaseItemView.swift
```

```
│   │   │   └── ItemRow.swift
│   │   ├── Services/
│   │   │   └── ItemService.swift
│   │   ├── Repositories/
│   │   │   ├── ItemRepository.swift
│   │   │   ├── NetworkItemRepository.swift
│   │   │   └── MockItemRepository.swift
│   │   ├── DataStores/
│   │   │   └── ItemDataStore.swift
│   │   └── Models/
│   │       ├── GiftItem.swift
│   │       └── ItemStatus.swift
│   │
│   ├── Statistics/
│   │   ├── Views/
│   │   │   ├── StatisticsView.swift
│   │   │   ├── BudgetChartView.swift
│   │   │   ├── SpendingChartView.swift
│   │   │   └── StatsFilterView.swift
│   │   ├── Services/
│   │   │   └── StatisticsService.swift
│   │   ├── Repositories/
│   │   │   ├── StatisticsRepository.swift
│   │   │   ├── NetworkStatisticsRepository.swift
│   │   │   └── MockStatisticsRepository.swift
│   │   └── Models/
│   │       ├── ListStats.swift
│   │       └── BudgetData.swift
│   │
│   ├── Profile/
│   │   ├── Views/
│   │   │   ├── ProfileView.swift
│   │   │   ├── EditProfileView.swift
│   │   │   ├── NotificationSettingsView.swift
│   │   │   └── AccountSettingsView.swift
│   │   ├── Services/
│   │   │   └── ProfileService.swift
│   │   ├── Repositories/
│   │   │   ├── ProfileRepository.swift
│   │   │   ├── NetworkProfileRepository.swift
│   │   │   └── MockProfileRepository.swift
│   │   ├── DataStores/
│   │   │   └── UserDataStore.swift
│   │   └── Models/
│   │       └── User.swift
│   │
│   └── Notifications/
│       ├── Services/
│       │   ├── NotificationService.swift
│       │   └── DeepLinkHandler.swift
│       ├── Repositories/
│       │   ├── NotificationRepository.swift
│       │   ├── NetworkNotificationRepository.swift
│       │   └── MockNotificationRepository.swift
│       └── Models/
│           ├── NotificationPreferences.swift
│           └── PushNotification.swift
│
├── DesignSystem/
│   ├── Colors/
│   │   └── AppColors.swift                # Color extensions
│   ├── Typography/
│   │   └── AppTypography.swift            # Font extensions
```

```
│   ├── Spacing/
│   │   └── AppSpacing.swift                # Spacing constants
│   ├── Components/
│   │   ├── Buttons/
│   │   │   ├── PrimaryButton.swift
│   │   │   ├── SecondaryButton.swift
│   │   │   └── TertiaryButton.swift
│   │   ├── Cards/
│   │   │   ├── ListCard.swift
│   │   │   ├── ItemCard.swift
│   │   │   └── StatsCard.swift
│   │   ├── Inputs/
│   │   │   ├── TextFieldStyle.swift
│   │   │   ├── SecureFieldStyle.swift
│   │   │   └── PickerStyle.swift
│   │   ├── Loading/
│   │   │   ├── LoadingView.swift
│   │   │   └── SkeletonView.swift
│   │   └── EmptyStates/
│   │       ├── EmptyStateView.swift
│   │       └── ErrorStateView.swift
│   └── Icons/
│       └── AppIcons.swift                  # SF Symbols mapping
│
├── Navigation/
│   ├── TabBarView.swift                    # Main tab bar
│   ├── NavigationCoordinator.swift         # Deep link handling
│   └── Route.swift                         # Navigation routes
│
├── Utilities/
│   ├── Extensions/
│   │   ├── Date+Extensions.swift
│   │   ├── String+Extensions.swift
│   │   ├── View+Extensions.swift
│   │   └── Color+Extensions.swift
│   ├── Helpers/
│   │   ├── DateFormatter+Shared.swift
│   │   ├── NumberFormatter+Shared.swift
│   │   └── ImageCache.swift
│   └── Validation/
│       ├── EmailValidator.swift
│       └── PasswordValidator.swift
│
├── Resources/
│   ├── Assets.xcassets/
│   │   ├── Colors/
│   │   │   ├── Primary/
│   │   │   ├── Secondary/
│   │   │   ├── Accent/
│   │   │   └── Neutral/
│   │   ├── Icons/
│   │   │   └── AppIcon.appiconset/
│   │   └── Images/
│   │       └── LaunchScreen.imageset/
│   ├── Localization/
│   │   └── en.lproj/
│   │       └── Localizable.strings
│   └── Info.plist
│
└── Preview Content/
    ├── PreviewData/
    │   ├── MockUser.swift
    │   ├── MockGiftList.swift
```

```
    │   └── MockGiftItem.swift
    └── PreviewHelpers/
        ├── PreviewContainer.swift
        └── MockEnvironment.swift
```

---

3. Test Target Structure

```
ChristmasTrackerTests/
├── Core/
│   ├── Networking/
│   │   ├── APIClientTests.swift
│   │   ├── NetworkCacheTests.swift
│   │   └── ErrorInterceptorTests.swift
│   ├── Security/
│   │   ├── KeychainServiceTests.swift
│   │   └── DeviceFingerprintServiceTests.swift
│   └── Logging/
│       ├── LoggerTests.swift
│       └── PIIFilterTests.swift
│
├── Features/
│   ├── Authentication/
│   │   └── AuthenticationServiceTests.swift
│   ├── Lists/
│   │   ├── ListServiceTests.swift
│   │   └── ListRepositoryTests.swift
│   └── Items/
│       ├── ItemServiceTests.swift
│       └── ItemRepositoryTests.swift
│
├── Mocks/
│   ├── MockAPIClient.swift
│   ├── MockNetworkCache.swift
│   ├── MockListRepository.swift
│   ├── MockItemRepository.swift
│   └── MockAuthRepository.swift
│
└── TestHelpers/
    ├── TestDataFactory.swift
    └── AsyncTestHelpers.swift
```

---

4. UI Test Target Structure

```
ChristmasTrackerUITests/
├── Flows/
│   ├── LoginFlowUITests.swift
│   ├── CreateListFlowUITests.swift
│   ├── AddItemFlowUITests.swift
│   └── PurchaseFlowUITests.swift
│
├── Helpers/
│   ├── XCUIApplication+Extensions.swift
│   └── XCUIElement+Extensions.swift
│
└── PageObjects/
    ├── LoginPage.swift
```

```
    ├── DashboardPage.swift
    ├── ListDetailPage.swift
    └── ItemDetailPage.swift
```

---

5. Share Extension Target Structure

```
ShareExtension/
├── ShareViewController.swift           # Extension entry point
├── Views/
│   ├── ShareFormView.swift            # Share UI
│   └── ListPickerView.swift           # List selection
├── Services/
│   └── ShareService.swift             # Share logic
├── Repositories/
│   ├── ShareRepository.swift          # Protocol
│   ├── NetworkShareRepository.swift
│   └── MockShareRepository.swift
├── Models/
│   ├── ShareItemRequest.swift
│   └── URLMetadata.swift
├── Utilities/
│   └── URLMetadataExtractor.swift     # URL parsing
├── Resources/
│   ├── Assets.xcassets/
│   └── Info.plist
└── ShareExtension.entitlements         # Keychain sharing
```

---

6. Configuration Files

```
Config/
├── Config.xcconfig                     # Production config
├── Config-Debug.xcconfig               # Debug config
└── Shared.xcconfig                     # Shared settings
```

Config.xcconfig (Production)

```
// Production configuration
IOS_DEPLOYMENT_TARGET = 26.0
SWIFT_VERSION = 6.0
PRODUCT_BUNDLE_IDENTIFIER = com.christmastracker.ios

// Custom settings
CACHE_DURATION = 900
SESSION_TIMEOUT = 7200
BACKGROUND_REFRESH = 900
LOG_FLUSH_INTERVAL = 60
```

Config-Debug.xcconfig

```
// Debug configuration
#include "Config.xcconfig"
```

```
// Override for debug
CACHE_DURATION = 30
SESSION_TIMEOUT = 300
BACKGROUND_REFRESH = 60
LOG_FLUSH_INTERVAL = 15
```

---

7. File Naming Conventions

7.1 Swift Files

Pattern: `PascalCase` for all file names

```
✅ Correct:
- AuthenticationService.swift
- LoginView.swift
- GiftList.swift
- NetworkListRepository.swift

❌ Incorrect:
- authenticationService.swift
- login_view.swift
- gift-list.swift
```

7.2 View Files

Pattern: `[Feature][ComponentType].swift`

```
✅ Examples:
- LoginView.swift
- DashboardView.swift
- ListCard.swift
- PrimaryButton.swift
- ItemRow.swift
```

7.3 Service Files

Pattern: `[Feature]Service.swift`

```
✅ Examples:
- AuthenticationService.swift
- ListService.swift
- ItemService.swift
- NotificationService.swift
```

7.4 Repository Files

Pattern:

- Protocol: `[Feature]Repository.swift`
- Network: `Network[Feature]Repository.swift`
- Mock: `Mock[Feature]Repository.swift`

```
✅ Examples:
- ListRepository.swift (protocol)
- NetworkListRepository.swift (network implementation)
- MockListRepository.swift (mock implementation)
```

7.5 DataStore Files

Pattern: `[Feature]DataStore.swift`

```
✅ Examples:
- ListDataStore.swift
- ItemDataStore.swift
- UserDataStore.swift
```

7.6 Model Files

Pattern: `[ModelName].swift` (singular noun)

```
✅ Examples:
- GiftList.swift
- GiftItem.swift
- User.swift
- ListMember.swift
```

7.7 Test Files

Pattern: `[FileName]Tests.swift`

```
✅ Examples:
- AuthenticationServiceTests.swift
- ListRepositoryTests.swift
- LoginFlowUITests.swift
```

---

8. Code Organization Guidelines

8.1 File Length

- **Maximum 400 lines** per file
- Split large files into logical components
- Extract complex logic into separate files

8.2 Type Organization

Within a file, organize in this order:

```swift
// 1. Imports
import SwiftUI
import Observation

// 2. Main Type
@MainActor
```

```swift
@Observable
class ListService {
    // 3. Properties
    private let repository: ListRepository
    var lists: [GiftList] = []
    var isLoading = false

    // 4. Initializers
    init(repository: ListRepository) {
        self.repository = repository
    }

    // 5. Public Methods
    func fetchLists() async {
        // ...
    }

    // 6. Private Methods
    private func updateCache() {
        // ...
    }
}

// 7. Extensions
extension ListService {
    func filter(by query: String) -> [GiftList] {
        // ...
    }
}

// 8. Supporting Types
enum ListFilter {
    case all
    case owned
    case shared
}
```

8.3 Import Order

```swift
// 1. System frameworks
import Foundation
import SwiftUI
import Observation

// 2. Third-party frameworks
import Sentry
import NewRelic

// 3. Internal modules
@testable import ChristmasTracker
```

---

9. Environment Files

9.1 Environment Keys

```swift
// Core/Configuration/Environment.swift
```

```swift
extension EnvironmentValues {
    // Services
    var authService: AuthenticationService {
        get { self[AuthServiceKey.self] }
        set { self[AuthServiceKey.self] = newValue }
    }

    var listService: ListService {
        get { self[ListServiceKey.self] }
        set { self[ListServiceKey.self] = newValue }
    }

    var itemService: ItemService {
        get { self[ItemServiceKey.self] }
        set { self[ItemServiceKey.self] = newValue }
    }

    // ... other services
}

// Environment keys
private struct AuthServiceKey: EnvironmentKey {
    static let defaultValue: AuthenticationService = {
        AuthenticationService(
            repository: NetworkAuthRepository(apiClient: .shared),
            keychainService: .shared,
            fingerprintService: .shared
        )
    }()
}

private struct ListServiceKey: EnvironmentKey {
    static let defaultValue: ListService = {
        ListService(
            repository: NetworkListRepository(
                apiClient: .shared,
                cache: .shared
            ),
            dataStore: ListDataStore()
        )
    }()
}
```

---

10. Asset Catalog Structure

```
Assets.xcassets/
├── Colors/
│   ├── Primary/
│   │   ├── primary-50.colorset
│   │   ├── primary-100.colorset
│   │   ├── primary-500.colorset      # Main brand color
│   │   └── primary-900.colorset
│   ├── Secondary/
│   │   ├── secondary-50.colorset
│   │   └── secondary-500.colorset
│   ├── Accent/
│   │   └── accent-500.colorset
│   ├── Neutral/
│   │   ├── neutral-50.colorset       # Light backgrounds
```

```
│   │   ├── neutral-100.colorset
│   │   ├── neutral-500.colorset      # Mid-tones
│   │   ├── neutral-900.colorset      # Text
│   │   └── neutral-950.colorset      # Dark backgrounds
│   ├── Semantic/
│   │   ├── success.colorset
│   │   ├── warning.colorset
│   │   ├── error.colorset
│   │   └── info.colorset
│   └── Surface/
│       ├── background.colorset
│       ├── surface.colorset
│       └── overlay.colorset
│
├── Icons/
│   ├── AppIcon.appiconset/
│   │   ├── Contents.json
│   │   ├── icon-1024.png
│   │   ├── icon-60@2x.png
│   │   └── ... (all required sizes)
│   └── TabBar/
│       ├── dashboard-icon.imageset/
│       ├── stats-icon.imageset/
│       └── profile-icon.imageset/
│
├── Images/
│   ├── LaunchScreen.imageset/
│   ├── EmptyStates/
│   │   ├── empty-lists.imageset/
│   │   └── empty-items.imageset/
│   └── Illustrations/
│       └── error-state.imageset/
│
└── Data/
    └── app-data.json                 # Mock data for previews
```

---

11. GitHub Actions Structure

```
.github/
└── workflows/
    ├── ios.yml                       # Main CI/CD workflow
    ├── lint.yml                      # SwiftLint checks
    └── security.yml                  # Security scanning
```

---

12. Scripts Directory

```
Scripts/
├── setup.sh                          # Initial project setup
├── generate-mocks.sh                 # Generate mock files
├── update-dependencies.sh            # Update SPM packages
└── build-archive.sh                  # Archive build script
```

---

13. Documentation Structure

```
Docs/
├── Architecture/
│   ├── 01_Architecture_Decision_Record.md
│   ├── 02_Project_Structure_Blueprint.md
│   ├── 03_Design_System_Specification.md
│   └── 04_Phase_1_Implementation_Guide.md
│
├── API/
│   ├── Authentication.md
│   ├── Lists.md
│   ├── Items.md
│   └── Notifications.md
│
└── Guides/
    ├── Setup_Guide.md
    ├── Testing_Guide.md
    └── Deployment_Guide.md
```

---

14. SPM Package Integration

14.1 Package.swift (if using local packages)

```swift
// swift-tools-version:5.9
import PackageDescription

let package = Package(
    name: "ChristmasTrackerCore",
    platforms: [
        .iOS(.v26)
    ],
    products: [
        .library(
            name: "ChristmasTrackerCore",
            targets: ["Networking", "Security", "Logging"]
        )
    ],
    dependencies: [
        .package(url: "https://github.com/getsentry/sentry-cocoa", from: "8.0.0")
    ],
    targets: [
        .target(name: "Networking"),
        .target(name: "Security"),
        .target(name: "Logging"),
        .testTarget(
            name: "NetworkingTests",
            dependencies: ["Networking"]
        )
    ]
)
```

14.2 Package Dependencies

```
Dependencies/
├── Sentry/                          # Crash reporting
├── NewRelic/ (if compatible)        # Observability
└── ... (other SPM packages)
```

15. .gitignore

```
# Xcode
*.xcodeproj/*
!*.xcodeproj/project.pbxproj
!*.xcodeproj/xcshareddata/
!*.xcworkspace/contents.xcworkspacedata
*.xcuserstate
*.xcuserdatad
DerivedData/
.build/

# Swift Package Manager
.swiftpm/
Package.resolved

# CocoaPods (if used)
Pods/
*.podspec

# Fastlane
fastlane/report.xml
fastlane/Preview.html
fastlane/screenshots
fastlane/test_output

# Code coverage
*.profdata
*.coverage

# Environment
.env
.env.local

# macOS
.DS_Store
.AppleDouble
.LSOverride

# IDEs
.vscode/
.idea/
*.swp
*.swo
*~
```

16. Code Organization Standards

16.1 File Length Limits

- **Maximum 400 lines** per file
- Split large files into logical components
- Extract complex logic into separate files
- Use extensions in separate files for large protocol conformances

16.2 Type Organization Within Files

Order of sections:

```swift
// 1. Imports (system, third-party, internal)
import SwiftUI
import Observation

// 2. Type Declaration & Properties
@MainActor
@Observable
class MyService {
    // MARK: - Properties
    private let repository: MyRepository
    var items: [Item] = []
    var isLoading = false

    // MARK: - Initialization
    init(repository: MyRepository) {
        self.repository = repository
    }

    // MARK: - Public Methods
    func fetchItems() async {
        // Implementation
    }

    // MARK: - Private Methods
    private func updateCache() {
        // Implementation
    }
}

// 3. Protocol Conformances (as extensions)
// MARK: - Equatable
extension MyService: Equatable {
    static func == (lhs: MyService, rhs: MyService) -> Bool {
        lhs.items == rhs.items
    }
}

// MARK: - CustomStringConvertible
extension MyService: CustomStringConvertible {
    var description: String {
        "MyService(items: \(items.count))"
    }
}

// 4. Supporting Types (if small)
enum MyServiceError: Error {
    case networkFailure
    case invalidData
}
```

16.3 Protocol Conformances

Base implementation vs Extensions:

Rule: Base declaration includes fundamental protocols (SwiftUI View, ObservableObject, MainActor), additional protocols go in extensions.

✅ **Correct - View/Actor is base, other protocols in extensions:**

```swift
// MyView.swift

import SwiftUI

struct MyView: View {  // View is fundamental, part of base
    let data: [Item]

    var body: some View {
        List(data) { item in
            Text(item.name)
        }
    }
}

// MARK: - Equatable
extension MyView: Equatable {
    static func == (lhs: Self, rhs: Self) -> Bool {
        lhs.data == rhs.data
    }
}
```

✅ **Correct - Large conformance in separate file:**

```swift
// MyService.swift - Base implementation
@MainActor
@Observable
class MyService {
    // Base implementation (< 400 lines)
}

// MyService+Codable.swift - Complex protocol conformance
extension MyService: Codable {
    enum CodingKeys: String, CodingKey {
        // ... many keys
    }

    func encode(to encoder: Encoder) throws {
        // ... complex encoding (200+ lines)
    }

    init(from decoder: Decoder) throws {
        // ... complex decoding
    }
}
```

16.4 SwiftUI View Components

Rule: Each component should be a standalone struct. Avoid nesting component definitions unless absolutely necessary.

✅ **Correct - Standalone components:**

```swift
// ListDetailView.swift

struct ListDetailView: View {
    let list: GiftList

    var body: some View {
        ScrollView {
```

```
            ForEach(list.items) { item in
                ItemRow(item: item)  // ✅ Separate component
            }
        }
    }
}

// Separate struct (same file is fine if small)
struct ItemRow: View {
    let item: GiftItem

    var body: some View {
        HStack {
            Text(item.name)
            Spacer()
            Text("$\(item.price)")
        }
    }
}
```

❌ **Incorrect - Nested components:**

```
struct ListDetailView: View {
    let list: GiftList

    var body: some View {
        ScrollView {
            ForEach(list.items) { item in
                // ❌ Avoid inline components
                HStack {
                    Text(item.name)
                    Spacer()
                    Text("$\(item.price)")
                }
            }
        }
    }
}
```

16.5 Import Organization

Order:

1. System frameworks
2. Third-party frameworks
3. Internal modules

```
// ✅ Correct order
import Foundation
import SwiftUI
import Observation

import Sentry

@testable import ChristmasTracker
```

16.6 Avoid Force Unwrapping

Never use `!` except in tests or for assets:

```
// ❌ Never
let url = URL(string: "https://api.example.com")!

// ✅ Use guard
guard let url = URL(string: urlString) else {
    fatalError("Invalid URL configuration")
}

// ✅ Acceptable for assets
let icon = UIImage(named: "app-icon")!  // Build error if missing
```

---

17. Best Practices

16.1 Feature Folder Organization

Each feature should be self-contained with all related code in one directory:

```
Features/Lists/
├── Views/           # UI components
├── Services/        # Business logic
├── Repositories/    # Data access
├── DataStores/      # In-memory cache
└── Models/          # Data structures
```

16.2 Avoid Deep Nesting

Maximum 3 levels of nesting:

```
✅ Good:
Features/Lists/Views/ListDetailView.swift

❌ Too deep:
Features/Lists/UI/Views/Detail/ListDetailView.swift
```

16.3 Group Related Files

Use Xcode groups (not folders) to organize related files:

```
Lists (Group)
├── Views (Group)
│   ├── ListDetailView.swift
│   └── ListCard.swift
└── Services (Group)
    └── ListService.swift
```

16.4 Shared Code Location

Code used across features goes in `Core/` or `Utilities/`:

```
```

```
Core/           # Essential infrastructure
Utilities/      # Helpers and extensions
DesignSystem/   # UI components
```

---

17. Xcode Project Configuration

17.1 Build Configurations

- **Debug:** Development builds with debug symbols
- **Release:** Production builds, optimized

17.2 Schemes

- **ChristmasTracker:** Main app scheme
- **ChristmasTracker (Testing):** Unit tests
- **ChristmasTracker (UI Testing):** UI tests
- **ShareExtension:** Share extension

17.3 Targets

- **ChristmasTracker:** Main iOS app
- **ChristmasTrackerTests:** Unit tests
- **ChristmasTrackerUITests:** UI tests
- **ShareExtension:** Share extension

---

18. Code Generation

18.1 Swagger/OpenAPI

Use `openapi-generator` or similar to generate API models:

```
# Scripts/generate-api-models.sh
openapi-generator generate \
  -i ../backend/tracker-swagger.json \
  -g swift6 \
  -o ./Generated/API \
  --additional-properties=useSwiftPackageManager=true
```

Output:

```
Generated/
└── API/
    ├── Models/
    │   ├── GiftList.swift
    │   ├── GiftItem.swift
    │   └── User.swift
    └── APIs/
        ├── AuthAPI.swift
        ├── ListsAPI.swift
        └── ItemsAPI.swift
```

18.2 Mock Generation

Use `Sourcery` or similar for automated mock generation:

```
# Scripts/generate-mocks.sh
sourcery \
  --sources ./ChristmasTracker \
  --templates ./Templates/Mocks.stencil \
  --output ./ChristmasTrackerTests/Mocks
```

---

19. Modularization Strategy

19.1 Current Structure (Monolithic)

All code in single app target (Phase 1-12)

19.2 Future Modularization (Post-MVP)

Split into Swift Packages:

```
Packages/
├── ChristmasTrackerCore/
│   ├── Networking
│   ├── Security
│   └── Logging
├── ChristmasTrackerFeatures/
│   ├── Authentication
│   ├── Lists
│   └── Items
└── ChristmasTrackerDesignSystem/
    ├── Colors
    ├── Typography
    └── Components
```

---

20. Summary

This project structure provides:

✅ **Clear organization:** Features grouped logically
✅ **Scalability:** Easy to add new features
✅ **Testability:** Tests mirror app structure
✅ **Consistency:** Naming conventions enforced
✅ **Maintainability:** Related code stays together

Next Steps:

1. Review this structure with team
2. Create initial folders in Phase 1
3. Document any deviations in this file

End of Project Structure Blueprint